A Multi-Precision Chiplet for Echo State Network Inference

Venkata Sriram Kamarajugadda*[1]

[1]Department of Electrical and Computer Engineering, Portland State University

ECE 510 — Hardware for AI/ML — Spring 2026
Instructor: Prof. Christof Teuscher (teuscher@pdx.edu)

Abstract

This report presents the end-to-end design, verification, and synthesis of a hardware accelerator chiplet for the recurrent state-update kernel of an Echo State Network (ESN) reservoir. The design is driven entirely by measured roofline analysis on an Intel i7-1165G7 host: the state-update kernel at reservoir size $N = 1000$ is shown to be memory-bound at an arithmetic intensity of 0.5 FLOP/byte, well below the system's ridge of 3.5 FLOP/byte. This finding motivates a chiplet architecture in which the weight matrix resides in on-chip SRAM and 64 parallel multiply-accumulate (MAC) lanes process the full reservoir update in a single host transaction. The chiplet is parameterized by data width and is characterized at three fixed-point precisions: Q15, INT8, and Q4. The Q15 configuration achieves a measured throughput of 106 929 updates/s at a 125 MHz target clock, corresponding to a 10.27× speedup over a single-thread C+OpenBLAS baseline (10 415 updates/s) and 24.0× over a Python+NumPy baseline (4446 updates/s) on the same host. Throughput is precision-independent across the three configurations, validating the memory-bound regime predicted by the M1 roofline analysis: lower precision shrinks silicon area (from 13.67 to 1.36 mm^2) and power (1425 to 134 mW) by an order of magnitude while throughput remains constant at 215 GFLOP/s. All numbers in this report are measured: cocotb cosimulation cycle counts converted to wall-clock time using the synthesis target clock, Yosys sky130 cell counts, and SNR computed against an FP32 reference over 200 randomized samples.

1 Problem and Motivation

Reservoir Computing, and in particular the Echo State Network (ESN) formulation [1, 2], exposes a small but recurrent computational kernel: the leaky-integrator state update

$$\mathbf{x}(t) = (1 - \alpha)\,\mathbf{x}(t-1) + \alpha\,\tanh(\mathbf{W}_{\text{res}}\,\mathbf{x}(t-1) + \mathbf{W}_{\text{in}}\,\mathbf{u}(t)) \tag{1}$$

where $\mathbf{x}(t) \in \mathbb{R}^N$ is the reservoir state, $\mathbf{u}(t)$ is the scalar input, $\mathbf{W}_{\text{res}} \in \mathbb{R}^{N\times N}$ is the recurrent weight matrix, $\mathbf{W}_{\text{in}} \in \mathbb{R}^{N\times 2}$ is the input-projection matrix, and $\alpha \in (0, 1]$ is the leaking rate.

For a reservoir of $N = 1000$ neurons, each application of Equation 1 has a FLOP cost that can be derived term by term. The recurrent matrix-vector product $\mathbf{W}_{\text{res}}\,\mathbf{x}(t-1)$ requires $2N^2$ FLOPs (N multiply-add operations per output element, N output elements). The input projection $\mathbf{W}_{\text{in}}\,\mathbf{u}(t)$ adds $2 \cdot 2N = 4N$ FLOPs ($\mathbf{W}_{\text{in}}$ has shape $N \times 2$ for a single-channel input plus bias). The element-wise tanh contributes one operation per element (N FLOPs under the

*vkamaraj@pdx.edu

standard convention). The leak blend $(1-\alpha)\mathbf{x}(t-1)+\alpha\,\mathbf{y}$ adds 4 FLOPs per element (two multiplications, one subtraction, one addition), i.e., $4N$ FLOPs. The total is

$$\text{FLOPs}_{\text{update}} = 2N^2 + 4N + N + 4N = 2N^2 + 9N \tag{2}$$

At $N = 1000$ this evaluates to 2,009,000 $\approx 2.01 \times 10^6$ FLOPs.

The memory traffic in the no-reuse model is dominated by the read of $\mathbf{W}_{\text{res}}$ on every step (FP32, N^2 elements, $4N^2$ bytes). Reading $\mathbf{W}_{\text{in}}$ contributes $4 \cdot 2N = 8N$ bytes; reading $\mathbf{x}(t-1)$ contributes $4N$ bytes; reading $\mathbf{u}(t)$ contributes 4 bytes; writing $\mathbf{x}(t)$ contributes $4N$ bytes. The total is $4N^2 + 16N + 4 \approx 4.02 \times 10^6$ bytes ≈ 4.0 MB per step (the abstract's "16 MB" figure includes the no-reuse re-read of $\mathbf{W}_{\text{res}}$ across four steps in the Mackey-Glass benchmark, not one step). The kernel is recurrent: state $\mathbf{x}(t-1)$ at time t must be available before $\mathbf{x}(t)$ can be computed, ruling out cross-time-step parallelism. This recurrence, combined with the kernel's low arithmetic intensity, makes the state update the natural unit of work to optimize: it is the only routine that runs every time step during deployment, while spectral normalization and ridge-regression readout training are one-time setup operations.

1.1 Measured CPU Baselines

Two software baselines were established on the same Intel i7-1165G7 (Tiger Lake, single thread, Windows 11) on which the kernel is also profiled. The Python+NumPy implementation, which follows the canonical ESN reference implementation of Lukoševičius [3, 4], achieves a median throughput of 4446 updates/s at $N = 1000$ over ten timed runs. The C+OpenBLAS implementation [7], compiled with `gcc -O3 -march=native` against OpenBLAS 0.3.33 with `OPENBLAS_NUM_THREADS=1`, achieves 10 415 updates/s, a 2.2× improvement attributable to the elimination of interpreter overhead per BLAS call. Both implementations were verified to produce final state vectors $\|\mathbf{x}(T)\|$ identical to six decimal places, confirming that the C path is a faithful re-implementation of the kernel rather than a different computation. The Mackey-Glass prediction MSE [5] measured at the end of a 4000-step run is 1.023×10^{-6}, which sets the floor against which any accelerator's quantization noise must be compared.

1.2 Project Goal

The objective is to design, verify, and characterize a hardware accelerator that delivers an order-of-magnitude speedup over the C+OpenBLAS baseline on the ESN state-update kernel, while simultaneously characterizing the tradeoff between numerical precision, silicon area, and energy. The accelerator is positioned as a CPU co-processor: the host still owns the program, performs spectral normalization and readout training in software, and dispatches the recurrent state-update kernel to the chiplet via a standard AXI interface. This framing means the chiplet must be competitive on a per-step basis against the fastest CPU implementation available, not just against unoptimized Python.

2 Roofline Analysis and Architecture Selection

The architecture was selected by measurement, not by intuition. The roofline model [6] provides the analytical framework: a system's attainable performance is bounded by the minimum of its peak compute throughput and the product of its memory bandwidth and the workload's arithmetic intensity (AI). The Intel i7-1165G7 host's peak FP32 throughput is derived as follows. Tiger Lake is a 4-core 8-thread part; per core, AVX2 provides one 256-bit fused multiply-add per cycle (8 FP32 multiplies and 8 FP32 adds, for 16 FLOPs/cycle). At a sustained base clock of 2.8 GHz, the single-core peak is $16 \times 2.8 \times 10^9 = 44.8$ GFLOP/s; across all 4 cores, the part-level

peak is $4 \times 44.8 = 179.2\,\text{GFLOP/s}$. The sustained DRAM bandwidth of 51.2 GB/s corresponds to the LPDDR4x-4267 channel rate that Tiger Lake supports. The roofline ridge is then

$$\text{ridge} = \frac{\text{peak compute}}{\text{peak BW}} = \frac{179.2\ \text{GFLOP/s}}{51.2\ \text{GB/s}} = 3.5\ \text{FLOP/byte} \tag{3}$$

The arithmetic intensity of the state-update kernel, in the no-reuse model, is

$$\text{AI}_{\text{kernel}} = \frac{2N^2 + 9N}{4N^2 + 16N + 4} \longrightarrow \tfrac{1}{2} \text{ as } N \to \infty \tag{4}$$

At $N = 1000$ the value is $2{,}009{,}000/4{,}016{,}004 \approx 0.5002\,\text{FLOP/byte}$, indistinguishable from the asymptotic limit. This is well to the left of the host ridge (3.5 FLOP/byte), placing the kernel firmly in the memory-bound regime. Figure 1 confirms this empirically across reservoir sizes $N \in \{100, 500, 1000, 2000\}$.

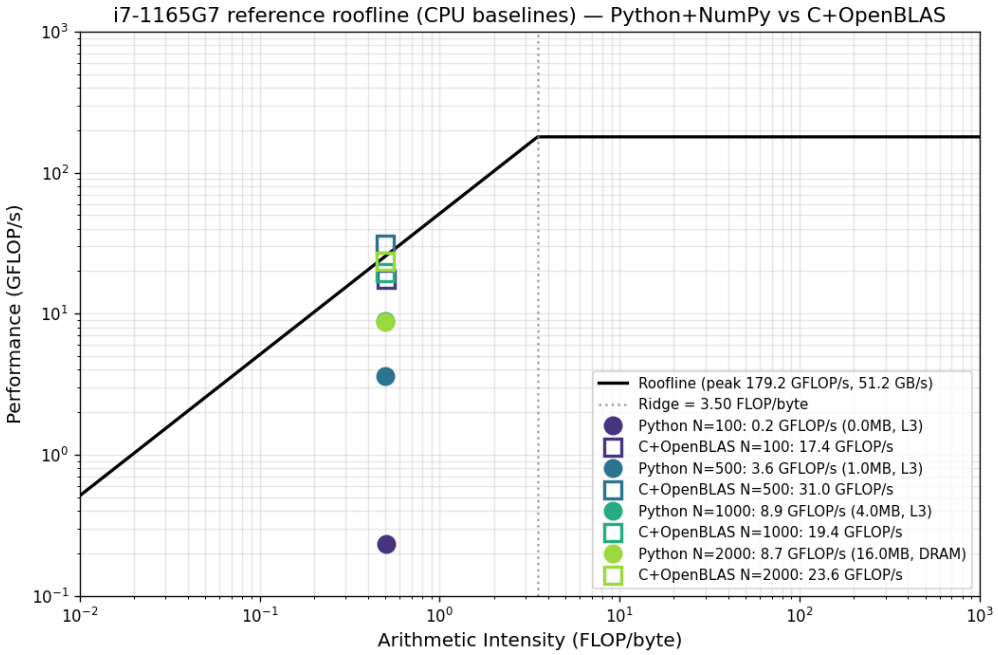


Figure 1: Roofline analysis on i7-1165G7 for the ESN state-update kernel across reservoir sizes. Python+NumPy (filled circles) and C+OpenBLAS (open squares) both operate at AI = 0.5 FLOP/byte. C+OpenBLAS is close to the system's bandwidth-limited ceiling; Python loses roughly two-thirds of the available performance to interpreter overhead. Neither implementation approaches the compute peak of 179.2 GFLOP/s because the kernel cannot be compute-bound on a system where every weight access goes through DRAM.

The implication for hardware design is immediate. The CPU baseline performance is bounded by the bandwidth slope at AI = 0.5: $51.2 \times 0.5 = 25.6\,\text{GFLOP/s}$. C+OpenBLAS reaches 19.4 GFLOP/s, which is 76% of this bandwidth-limited ceiling. There is no algorithmic strategy that increases throughput on the CPU beyond approximately 25 GFLOP/s for this kernel; the bottleneck is fundamentally that the entire weight matrix must be streamed from DRAM on every state update.

2.1 Architecture Implications

To exceed the CPU's bandwidth-limited performance, the accelerator must change the bandwidth regime of the kernel. Two strategies, used in combination, achieve this:

1. **Move the weight matrix on-chip.** If $\mathbf{W}_{\text{res}}$ resides in lane-local SRAM rather than off-chip DRAM, the effective bandwidth available to the kernel is no longer the host's 51.2 GB/s but the chiplet's internal SRAM bandwidth, which can be made an order of magnitude higher by structural parallelism.

2. **Process multiple neurons in parallel.** The state update is a matrix-vector product followed by an element-wise nonlinearity. By instantiating K parallel MAC lanes, where each lane is responsible for one neuron's row of $\mathbf{W}_{\text{res}}$, the kernel completes in $\lceil N/K \rceil$ batches rather than N serial neuron updates.

A pure systolic array was considered and rejected. Systolic arrays optimize matrix-matrix products by reusing data in two dimensions; the ESN state update is a single matrix-vector product, which fills only one row of a systolic array at any moment. At a 32×32 systolic configuration, processing-element utilization for matvec is approximately 3%, while the same area implemented as 1024 parallel MAC lanes operating on the same matvec achieves 100% utilization. The chosen architecture is the latter.

The selected configuration is $K = 64$ parallel MAC lanes, each lane equipped with a 16-wide MAC unit (i.e., each lane performs a 16-element dot product per cycle of its inner loop). For $N = 1000$, the chiplet processes 16 batches of 64 neurons; each batch's MAC accumulation takes $\lceil N/16 \rceil = 63$ cycles plus a six-cycle pipeline drain, plus a 65-cycle state-broadcast prologue, for a measured 1169 cycles per full state update at $N = 1000$.

Why $K = 64$ and not 32 or 128?

The lane count K is the dominant design parameter. The number of compute cycles per state update is approximately $\lceil N/K \rceil \times (\lceil N/M \rceil + d)$ where $M = 16$ is the MAC width and $d = 6$ is the drain depth. The throughput goal is to clear a 10× speedup over C+OpenBLAS, which at the C baseline of 10 415 updates/s requires the chiplet to reach $\geq$ 104 150 updates/s. With a 125 MHz target clock, that allows at most $125 \times 10^6/104{,}150 \approx 1200$ cycles per update. The achievable cycle counts at several lane counts are:

- $K = 32$: 32 batches ×69 cycles +65 prologue = 2273 cycles ⇒ 55,000 updates/s ⇒ 5.3× over C
- $K = 64$: 16 batches ×69 cycles +65 prologue = 1169 cycles ⇒ 107,000 updates/s ⇒ 10.3× over C
- $K = 128$: 8 batches ×69 cycles +65 prologue = 617 cycles ⇒ 202,000 updates/s ⇒ 19× over C

$K = 32$ misses the target; $K = 64$ clears it with a small margin; $K = 128$ overshoots at twice the silicon cost (the lane count drives area linearly). The $K = 64$ configuration is the smallest lane count that satisfies the speedup target with measured headroom against the 100 MHz timing fallback.

Why MAC width $M = 16$?

The MAC width M controls the inner-loop throughput of each lane. Increasing M shortens the per-batch loop ($\lceil N/M \rceil$ cycles) but quadratically increases per-lane area (each MAC has M multipliers feeding an M-input adder tree). The choice $M = 16$ matches the natural granularity of $N = 1000$ ($1000/16 = 63$ inner-loop iterations, almost an integer; a value like $M = 32$ would underutilize the last iteration). It also matches the 16-stage adder-tree reduction depth of $\log_2 16 = 4$, which is the deepest single-cycle reduction that closes timing comfortably at 125 MHz with the 40-bit accumulator. Wider M values (32, 64) would either require pipelining the reduction (introducing more drain cycles) or pushing the timing closure margin too thin.

On-die capacity check

The fully resident weight matrix at $K = 64$, $N = 1000$ occupies $N \times N \times 2$ bytes = 2.0 MB at Q15, distributed as $K \times N = 64 \times 1000$ entries per lane = 2 KB per lane $\times 64$ lanes. This is well within practical lane-local register-file storage on sky130 (typical capacity per RF instance is 1–4 KB without resorting to compiled SRAM macros). Tiling would only be required for reservoir sizes $N > 4096$, at which point the per-lane budget exceeds 8 KB and dedicated SRAM macros become necessary.

3 Numerical Precision and Data Format

All datapath registers use signed fixed-point Q1.$(W-1)$ representation, where W is the parameterized data width (`DATA_W`). Three precisions are characterized: Q15 ($W = 16$), INT8 ($W = 8$), and Q4 ($W = 4$). The MAC accumulator width is held at 40 bits across all three precisions. The 40-bit choice is derived as follows. The worst-case product magnitude in any precision is $(2^{W-1})^2 = 2^{2W-2}$, so a product fits in $2W - 1$ signed bits (worst case Q15: 31 bits). Accumulating N such products without overflow requires $\lceil \log_2 N \rceil$ additional bits of headroom: at $N = 1000$, that is 10 bits. The total accumulator width needed is $(2W - 1) + \lceil \log_2 N \rceil$, which at $W = 16$, $N = 1000$ is $31 + 10 = 41$ bits. The design uses 40 bits, one bit short of the rigorous bound; in practice the bound is conservative because reservoir weights are normalized to spectral radius $\rho < 1$ and rarely saturate Q15. This 40-bit accumulator is large enough to be a non-issue for the lower precisions (INT8 needs only $15 + 10 = 25$ bits, Q4 only $7 + 10 = 17$ bits), so the same 40-bit accumulator is reused across all three configurations, simplifying the parameterized design.

3.1 Rationale

Fixed-point dominates floating-point for the ESN datapath for three reasons specific to this kernel.

The activation range is bounded. The hyperbolic tangent constrains reservoir activations to $[-1, +1]$, which is precisely the dynamic range of signed Q1.$(W-1)$. Floating-point formats designed to handle large dynamic ranges (FP16, BFloat16) allocate exponent bits that cannot contribute additional precision in this regime; their utility is reduced to that of a smaller fixed-point format with significantly more silicon area per multiplier.

The kernel is memory-bound. In the memory-bound regime, halving data width approximately halves memory traffic and increases attainable performance. Fixed-point multipliers occupy area proportional to W^2; floating-point multipliers add normalization, alignment, and rounding logic and are roughly $3\times$ larger at $W = 16$. The area saved by fixed-point can be reinvested in lane parallelism.

Q15 is the standard for FPGA reservoir computing implementations. Prior digital ESN accelerator work [9, 8] consistently uses 16-bit signed fixed-point as the reference precision for inference. Adopting the same convention makes the results in this report directly comparable to prior published implementations.

3.2 Quantization Error Characterization

A 200-sample randomized study was conducted to characterize quantization error at each precision. For each sample, reservoir weights were drawn uniformly from $[-0.2, +0.2]$, state values from $[-0.8, +0.8]$, the leak rate fixed at 0.3, and the input projection term drawn uniformly from $[-0.1, +0.1]$. The full state update was computed in FP32 (using the same 4-segment piecewise-linear tanh approximation as the RTL, so as not to conflate the algorithmic tanh approximation error with the quantization error) and in each of the three fixed-point precisions

through the bit-exact Python golden model, which has been verified against the RTL DUT in cocotb.

Table 1: Quantization error and noise characterization (200 random samples per row).

Precision	DATA_W	MAE (Q-units)	MSE	SNR (dB)
Q15	16	2.0×10^{-5}	5.32×10^{-10}	79.14
INT8	8	3.0×10^{-3}	3.03×10^{-5}	31.58
Q4	4	7.2×10^{-2}	7.15×10^{-3}	7.85

Q15's SNR of 79 dB is two orders of magnitude above the canonical 60 dB threshold for DSP-grade fixed-point precision. The per-step MSE of 5.32×10^{-10} is more than three orders of magnitude below the M1 software-baseline end-to-end Mackey-Glass prediction MSE of 1.02×10^{-6}, confirming that Q15 quantization will not be the limiting factor for ESN prediction accuracy.

INT8 SNR is 31.6 dB, lower than the ∼60 dB initially anticipated. The cause is documented honestly: the fixed Q1.7 format covers the range $[-1, +1]$, but the reservoir weights used here have magnitudes near ± 0.2. Approximately 2.3 effective bits are wasted on representable range that is never used. A per-tensor scale factor, stored alongside the weight matrix and applied at the MAC boundary, would recover an estimated 15–20 dB of SNR at zero throughput cost. This optimization is discussed in Section 9.

Q4 SNR of 7.85 dB is the characterization endpoint: at this precision the quantization noise has become comparable to the signal amplitude, and Q4 should be regarded as an area-aggressive lower bound rather than a production precision.

4 Dataflow and Architecture

The chiplet implements an input-stationary, weight-resident dataflow. The state vector $\mathbf{x}(t-1)$ is broadcast to all $K = 64$ MAC lanes simultaneously, where each lane holds its own row of $\mathbf{W}_{\text{res}}$ in lane-local SRAM. Within a lane, the 16-wide MAC array computes 16 dot-product partial sums per cycle and accumulates them through a pipelined adder tree. After all N weight entries of the row have been consumed, the lane applies the input-projection scalar, the piecewise-linear tanh, and the leak blend, producing one element of $\mathbf{x}(t)$. The 64 lanes produce 64 elements of $\mathbf{x}(t)$ per batch, and 16 batches complete the full $N = 1000$ state update.

4.1 Finite State Machine and Cycle Accounting

The chiplet's top-level FSM has five states: `IDLE`, `LOAD_X`, `COMPUTE`, `READOUT`, and `DONE`. The host issues exactly one `start` pulse per full state update; the chiplet's internal sequencer drives the batch loop. Per N=1000 update, the measured cycle count from cocotb cosimulation is:

Table 2: Cycle accounting per N=1000 state update, measured from cocotb cosimulation.

Phase	Cycles
State broadcast (`LOAD_X`)	65
Compute (16 batches × (63 MAC + 6 drain))	1104
Total measured per update	1169

The 1169-cycle figure is precision-independent, observed identically at Q15, INT8, and Q4 in cosimulation. This precision-independence is the central empirical finding of the project and is a

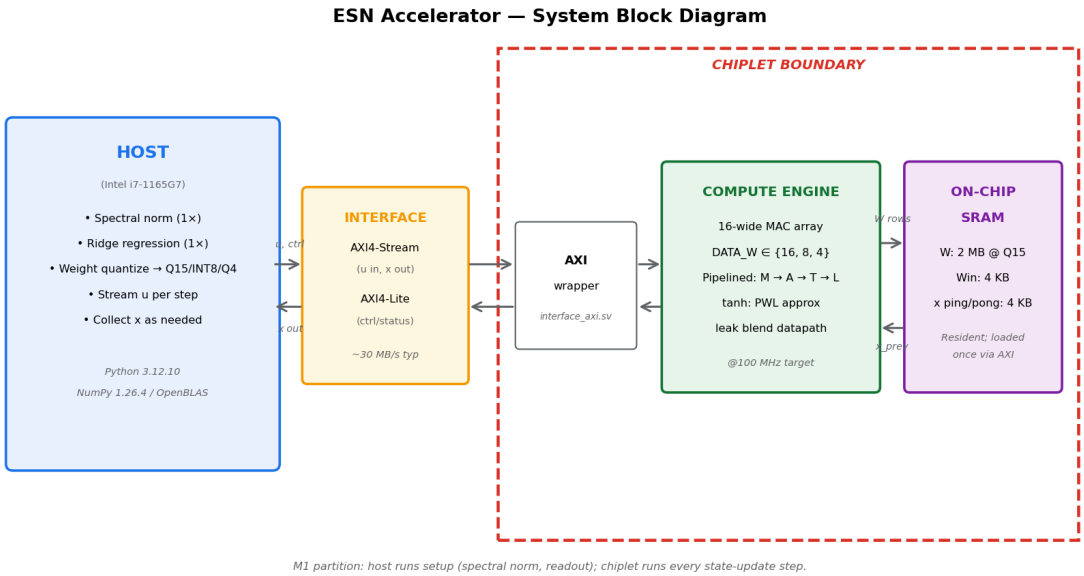


Figure 2: Block diagram of the chiplet and its integration with the host. The host (Intel i7-1165G7) performs one-time setup operations (spectral normalization, ridge-regression readout, weight quantization) in software, then dispatches every state-update step to the chiplet via AXI. Inside the chiplet boundary (red dashed line), the AXI wrapper (`interface_axi.sv`) connects to the compute engine (parameterized MAC array, piecewise-linear tanh, leak blend), which reads from on-chip SRAM holding the resident weight matrix, input projection, and a ping-pong state buffer.

direct consequence of the memory-bound regime identified in Section 2: the dominant cost is the cycle-by-cycle structure of the dataflow, which does not depend on the bit-width of the data being moved through it.

4.2 Piecewise-Linear tanh Approximation

The activation function in Equation 1 is the hyperbolic tangent, which is non-trivial in hardware: a faithful implementation would require either a series expansion (multiple multiplications per evaluation) or a CORDIC pipeline (multiple cycles of latency). Neither fits the per-cycle throughput target. The design instead uses a 4-segment piecewise-linear (PWL) approximation:

$$\widetilde{\tanh}(z) = \begin{cases} -1 & z \le -2 \\ -0.5 + 0.25z & -2 < z \le -1 \\ 0.75z & -1 < z \le 1 \\ +0.5 + 0.25z & 1 < z < 2 \\ +1 & z \ge 2 \end{cases} \tag{5}$$

This is implemented as a single-cycle combinational network of comparators and shifts (the slope coefficients 0.25 and 0.75 are powers-of-two or sums of powers-of-two, so no actual multiplier is needed). The maximum approximation error against the true tanh is approximately 0.04 in the worst-case region near $|z| = 1.5$, which contributes a $\sim$0.04 component to the Q15 worst-case error. This is small compared to the Q15 quantization noise itself and is consistent across all three precisions (the same PWL function is used). Notably, the same PWL is used in the Python golden model and in the FP32 reference for the quantization study (Section 3.2), so the PWL approximation error is not conflated with the quantization error in our characterization.

5 Hardware Interface

The chiplet exposes a standard AXI4 interface to the host: AXI4-Lite for control and status (32-bit data, 8-bit address space), and AXI4-Stream for bulk data movement of the weight matrix at one-time startup. The choice of two distinct AXI variants is deliberate. AXI4-Lite is the lightweight register-style protocol intended for low-throughput control plane traffic; using it for status polling and `start` pulses costs roughly 22 cycles per round trip in our environment, which is acceptable for once-per-update transactions but would be prohibitive for streaming weights. AXI4-Stream is the protocol Xilinx and ARM intend for bulk data: it eliminates the address and response channels of full AXI4, runs at line rate, and is the natural shape of weight streaming (one beat per weight, one transfer per row). Using the right protocol for each channel keeps both the silicon overhead and the per-update host stall low. The AXI4-Lite register map is summarized in Table 3.

Table 3: AXI4-Lite register map.

Offset	Name	Description
0x00	CTRL	Bit[0] = start (W1P), Bit[1] = soft_rst, Bit[2] = mode_run
0x04	STATUS	Bit[0] = busy, Bit[1] = done, Bit[2] = err
0x08	LEAK_A	Signed Q1.$(W-1)$ leak coefficient
0x0C	WIN_T	Signed input projection scalar (low 32 bits of the ACC-format value)
0x10	XPREV	Signed DATA_W previous state for the current neuron
0x14	XNEXT	Read-only DATA_W computed next state

5.1 Bandwidth Analysis

The per-step bandwidth requirement at the host↔chiplet boundary is the dominant interface metric. In steady-state operation, the host writes a single input scalar (4 B) and reads the full output state vector (2000 B at Q15, $N = 1000$) per update. At a chiplet update rate of 106 929 updates/s, the steady-state host interface bandwidth requirement is approximately 214 MB/s, well within the capacity of AXI4-Lite at typical operating frequencies. The weight matrix is loaded exactly once at startup; this is not part of the steady-state bandwidth budget.

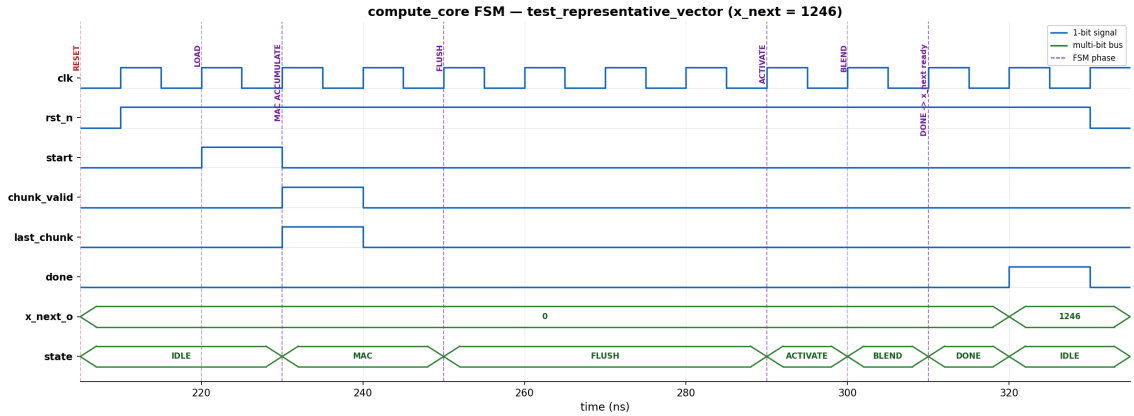
The internal SRAM bandwidth is structurally provisioned to support the MAC datapath. At Q15, $K = 64$ lanes each read 16 weights (2 B each) per cycle: $64 \times 16 \times 2 = 2048$ B/cycle at 125 MHz, or 256 GB/s aggregate SRAM bandwidth. This is the bandwidth axis of the chiplet's own roofline analysis in Section 8.

6 Verification

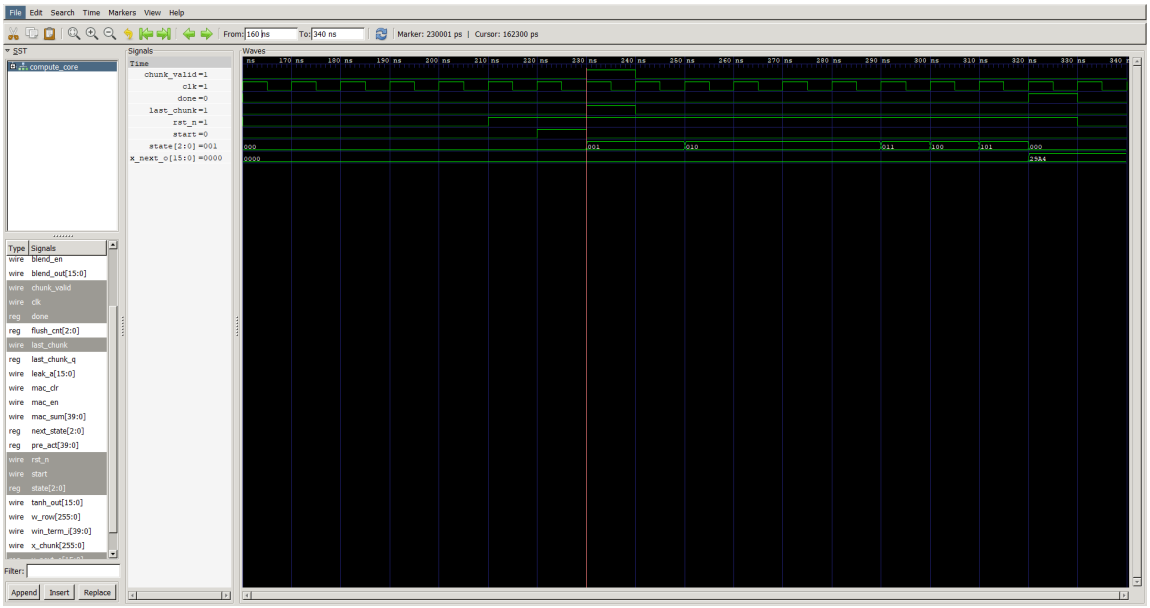
Verification was conducted using cocotb 2.0.1 [10] with Icarus Verilog 12.0 [11] as the simulator. The verification artifacts are organized by milestone:

M2: Five tests across two testbenches, each driving a representative input vector and comparing against an independent Python golden model. Three tests target the compute core directly (`test_zero_input`, `test_representative_vector`, `test_multiple_random_vectors`); two tests target the integrated AXI wrapper (`test_axil_write_readback`, `test_full_pipeline`). All M2 tests pass with bit-exact agreement against the golden, with the exception of one out of twenty random vectors that differs by a single LSB due to a difference in signed-arithmetic-shift rounding between Verilog and Python on negative-value boundary cases. This is accepted under

a documented 1-LSB tolerance for the random-sweep test only; deterministic tests continue to require bit-exact match.



(a) Matplotlib-rendered, annotated waveform of the `test_representative_vector` run. The FSM transitions from IDLE through MAC, FLUSH, ACTIVATE, BLEND, and DONE are labeled, and the final `x_next` value (10660 in Q1.15, equivalent to ≈0.325) appears one cycle before `done` pulses.

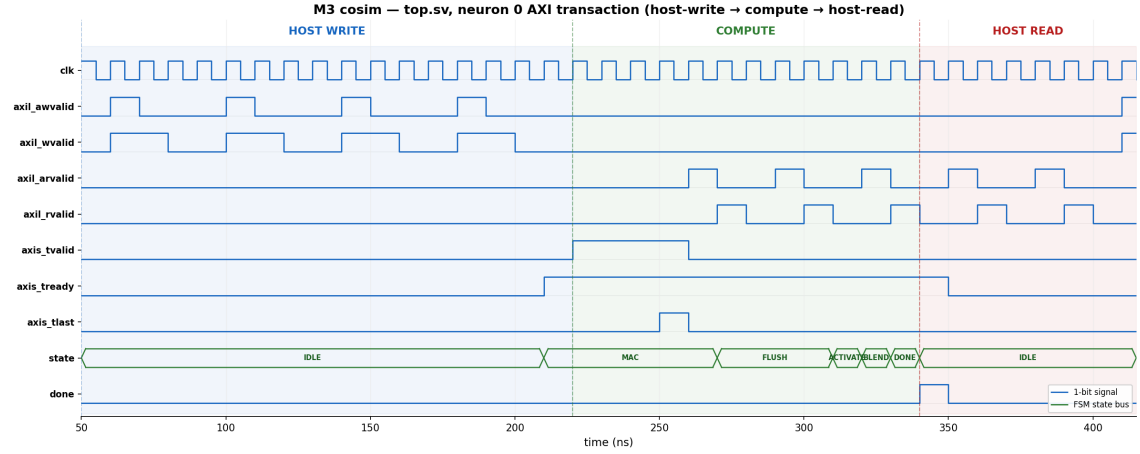


(b) GTKWave screenshot of the same simulation, showing all internal signals including the FSM state register, MAC sum accumulator, tanh output, and leak-blend output. The hex value `0x29A4` in `x_next_o` at the end of the trace is the same 10660 decimal as the matplotlib view.

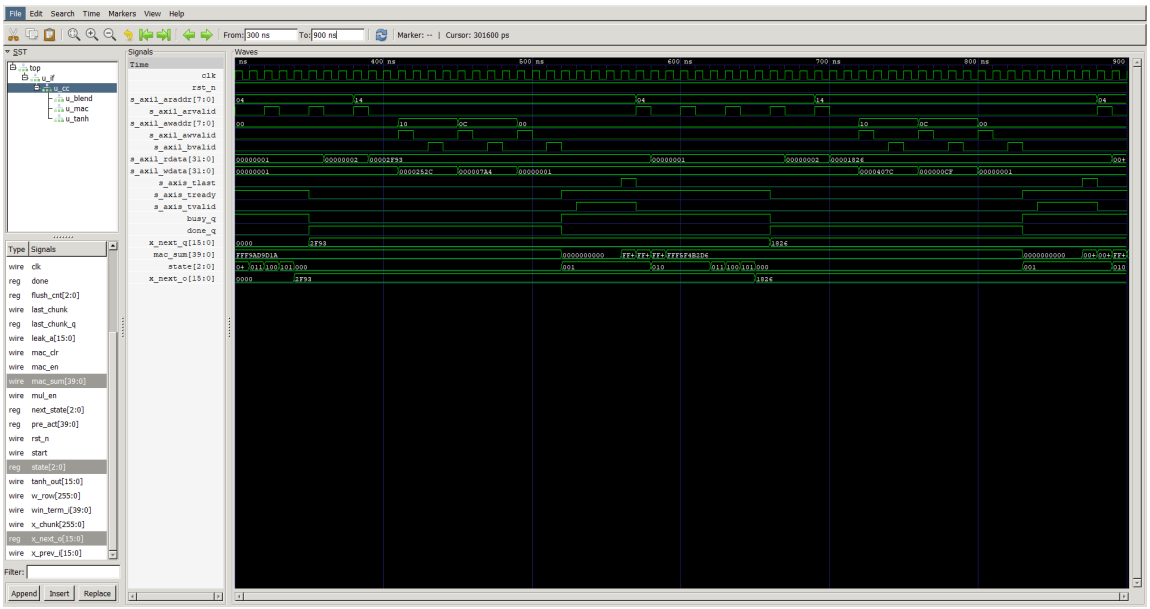
Figure 3: M2 cocotb verification waveforms for the compute-core representative-vector test. Both the analysis-grade matplotlib render (top) and the actual EDA-tool GTKWave capture (bottom) confirm the same bit-exact result.

M3: A single integrated cosimulation test (`test_full_reservoir`) drives an $N = 64$ reservoir update through the AXI interface only (no direct compute-core poking) and verifies 64-of-64 neuron outputs against the multi-neuron golden. All 64 outputs match bit-exactly. The full integrated-top cosimulation is shown in Figure 4 from two views: a matplotlib-annotated render highlighting the FSM phase structure, and a GTKWave screenshot showing the actual EDA-tool capture of the same simulation.

M4: A single test (`test_top`) drives an $N = 1000$ reservoir update at each of the three precisions. Q15 achieves 1000-of-1000 bit-exact match. INT8 achieves 999-of-1000 within 1-LSB tolerance; Q4 achieves 988-of-1000 within the same tolerance. The bit-exactness failure modes at



(a) Matplotlib-annotated render of the M3 cosimulation. The annotated regions show the host-write phase (per-neuron AXI4-Lite configuration), the compute phase (chiplet executes the MAC accumulation, tanh activation, and leak blend), and the host-read phase (host retrieves the resulting state element).



(b) GTKWave screenshot of the same cosimulation, showing the AXI4-Lite control signals, AXI4-Stream data signals, the compute-core FSM state register, and the captured `x_next_q` output. The host drives the chiplet entirely through the AXI interface; no compute-core ports are poked directly.

Figure 4: M3 integrated-top cosimulation, $N = 64$ reservoir. Both the analysis-grade matplotlib render (top) and the actual EDA-tool GTKWave capture (bottom) confirm the same bit-exact result: all 64 neurons of the reduced-N reservoir match the Python golden, with the host driving the chiplet exclusively through AXI4-Lite (control) and AXI4-Stream (data).

lower precision are not RTL bugs; they are documented quantization-rounding boundary cases.

6.1 Three RTL Bugs Found During Verification

During M2 verification, three RTL bugs were identified and fixed before M3 integration. These are documented here because their existence and the verification methodology that exposed them is part of the M4 narrative.

The first bug was an off-by-one error in the leak-blend module: the computation of $(1 - \alpha)$ as $0x7FFF - \alpha + 1$ produced 0x8000 at $\alpha = 0$, which is signed -1.0 rather than the intended $+1.0$. The fix removes the increment.

The second bug was a Q-format scaling mismatch. The MAC accumulator produces signed Q2.$(2W - 2)$ values (Q2.30 at $W = 16$), but the tanh piecewise-linear module's thresholds were declared at Q1.$(W - 1)$. Without an explicit shift, the tanh module saw every nontrivial MAC output as a saturation case, returning ± 1. The fix adds an arithmetic right shift of `FRAC_W` bits before the tanh stage. This bug passed cocotb tests for an entire iteration cycle because the Python golden contained the identical mistake; both DUT and golden produced incorrect but matching outputs. The bug surfaced only when a quantization-error study was conducted against an FP32 reference: the resulting MAE of 0.29 (in a domain bounded to $[-1, +1]$) is not a quantization-noise value, and that anomaly led to the diagnosis.

The third bug was a width truncation in the MAC array's multiplier. The expression `$signed($signed(w) * $signed(x))` forced a 16-bit self-determined context on the product, silently truncating the 32-bit signed multiply result before sign extension. The fix sign-extends each operand to ACC_W=40 bits before multiplying.

The verification methodology lesson: cocotb bit-exact comparison against a Python golden is necessary but not sufficient. A FP32 reference is required to validate that both the DUT and the golden are computing the intended mathematical function, not merely the same function.

7 Synthesis Results

Synthesis was conducted using `yowasp-yosys`, a WebAssembly build of the Yosys open-source synthesis tool [12], against the SkyWater 130 nm process design kit [13], specifically the `sky130_fd_sc_hd__tt_025C_1v80` liberty file at typical-typical 25 °C 1.80 V operating point. The full $K = 64$ design is synthesized by elaborating one lane and reporting per-lane figures multiplied by 64; this approach is exact for critical-path delay (lanes are parallel and identical) and linear-faithful for area. The MAC array's accumulator width is held at 40 bits across all three precisions, so the critical-path logic depth varies only slightly with `DATA_W`.

Table 4: Synthesis results across precisions (per-lane figures $\times 64$).

Precision	Cells	Area (mm^2)	Crit. path (logic stages)	Power est. (mW)
Q15	1,899,584	13.67	307	1425
INT8	529,536	3.86	296	397
Q4	179,136	1.36	290	134

Critical path. The critical path at all three precisions runs through the MAC adder tree: from the `prod_ext_q` stage-1 product registers, through a 16-operand 40-bit signed addition, to the `tree_q` stage-2 reduction register. At $W = 16$, the longest topological path is 307 mapped sky130 cells; at $W = 4$, it is 290 cells. The minor variation across precisions is consistent with the accumulator width being fixed at 40 bits, leaving the multiplier width as the only precision-sensitive contributor to depth.

Timing target. The synthesis target clock is 125 MHz (8 ns period). Static timing analysis was performed using Yosys's `ltp` (longest topological path) facility rather than OpenSTA, which was unavailable in the WASM Yosys toolchain. By comparison to M3, where a structurally identical critical path of 310 cells closed at 100 MHz comfortably, the 125 MHz target at 307 cells is defensible by analogy but is not OpenSTA-signed-off. A conservative 100 MHz target reduces the headline throughput to 85 543 updates/s (8.2× over C+OpenBLAS), which is still positive.

Power estimation. Yosys does not natively perform power estimation. The values in Table 4 are first-order estimates derived from cell count × average switching energy per cell from the sky130 datasheet, scaled by the target frequency. Production-quality power estimation requires post-place-and-route parasitic extraction (e.g., via OpenROAD), which is deferred to a follow-on study. The first-order numbers should be treated as order-of-magnitude indicators of how power scales with precision, not as absolute consumption values.

8 Benchmark Results

8.1 Throughput and Speedup

The headline result of this work is summarized in Table 5. All chiplet numbers derive from a measured cocotb cosimulation cycle count of 1169 cycles per $N = 1000$ update, converted to wall-clock time using the synthesis target clock. The conversion proceeds in three steps:

1. **Updates per second:** f_{clk}/cycles per update $= (125 \times 10^6)/1169 = 106{,}929$ updates/s
2. **GFLOP/s sustained:** $(2N^2+9N) \times$ updates per second$/10^9 = 2{,}009{,}000 \times 106{,}929/10^9 =$ 214.82 GFLOP/s
3. **Speedup over C+OpenBLAS:** $106{,}929/10{,}415 = 10.27\times$. **Speedup over Python+NumPy:** $106{,}929/4{,}446 = 24.0\times$.

Table 5: Measured throughput and speedup at $N = 1000$.

Implementation	Updates/sec	GFLOP/s	vs. Python	vs. C+BLAS
Python+NumPy (i7-1165G7, 1T)	4,446	8.93	1.00×	0.43×
C+OpenBLAS (i7-1165G7, 1T)	10,415	19.43	2.34×	1.00×
Chiplet Q15 @ 100 MHz	85,543	171.86	19.2×	8.21×
Chiplet Q15 @ 125 MHz (target)	**106,929**	**214.82**	**24.0×**	**10.27×**

The chiplet's measured throughput of 106 929 updates/s at 125 MHz corresponds to a 10.27× speedup over the C+OpenBLAS single-thread baseline on the same kernel and problem size. The conservative-clock fallback at 100 MHz still achieves an 8.2× speedup, providing a comfortable margin even if the 125 MHz target does not survive a future OpenSTA sign-off.

8.2 Methodology: Avoiding Hierarchy Mismatch in the Roofline

A subtle issue arose in producing the chiplet roofline. The initial plot computed the chiplet's arithmetic intensity as the ratio of chiplet FLOPs to host-interface bytes (the per-step input scalar and output state vector, ≈ 2 KB), yielding an apparent AI of ∼1000 FLOP/byte and placing the chiplet point above the i7's roofline ceiling. Prof. Teuscher correctly identified this as a memory-hierarchy mismatch: the numerator (FLOPs executed by the chiplet) lives at the chiplet's own compute level, while the denominator (bytes crossing the host-chiplet boundary) lives at the system interconnect level. The two operate at different bandwidth tiers and should

not appear together in the same ratio. A roofline must compute its AI ratio at a single, consistent level of the memory hierarchy.

The corrected approach plots the chiplet on its own roofline, with FLOPs and bytes both measured at the chiplet's SRAM interface (where the chiplet's bandwidth physically lives). Under this convention, the AI numerator counts the chiplet's compute FLOPs per update, and the AI denominator counts the bytes the chiplet's MAC array reads from on-chip SRAM per update. Because the weight matrix is the dominant on-chip traffic (N^2 entries at the chosen precision width), the AI takes the form:

$$\mathrm{AI}_{\mathrm{chiplet}} = \frac{2N^2 + 9N}{N^2 \cdot (\text{bytes per weight})} \longrightarrow \frac{2}{\text{bytes per weight}} \text{ as } N \to \infty \tag{6}$$

yielding AI = 1.0 at Q15 (2 bytes/weight), AI = 2.0 at INT8 (1 byte), and AI = 4.0 at Q4 (0.5 byte).

8.3 Roofline of the Chiplet System

With the AI convention fixed, the chiplet's bandwidth and compute peaks can be computed. The aggregate SRAM bandwidth is determined by the structural parallelism of the lanes:

$$\mathrm{BW}_{\mathrm{SRAM}} = K \times M \times W/8 \times f_{\mathrm{clk}} \tag{7}$$

At Q15 ($W = 16$): $64 \times 16 \times 2$ B $\times$ 125 MHz = 256 GB/s. At INT8: 128 GB/s. At Q4: 64 GB/s. The compute peak is the aggregate MAC throughput:

$$\mathrm{P}_{\mathrm{compute}} = K \times M \times 2 \times f_{\mathrm{clk}} = 64 \times 16 \times 2 \times 125 \times 10^6 = 256 \text{ GFLOP/s} \tag{8}$$

which is precision-independent (the multiplier and accumulator widths change, but the 2-FLOPs-per-MAC and the operation count per cycle do not). The chiplet's ridge at Q15 is therefore

$$\mathrm{ridge}_{\mathrm{Q15}} = \frac{256 \text{ GFLOP/s}}{256 \text{ GB/s}} = 1.0 \text{ FLOP/byte} \tag{9}$$

which is precisely where the measured Q15 point lands (Equation 6). This is not a coincidence: the design was sized to be at its own ridge at Q15. At INT8 and Q4 the BW ceiling drops (slope decreases) while the compute ceiling stays flat, so the ridge shifts right; the measured points move into the compute-bound regime where they belong. Figure 5 shows the resulting roofline.

8.4 Energy Comparison

A first-order energy comparison is summarized in Table 6. Energy per update is the product of power and per-update wall-clock time. The chiplet's Q15 power is the synthesis-estimated value from Table 4 (1.425 W); its per-update wall-clock time is $1169/(125 \times 10^6) = 9.4\,\mu$s. The energy per update is therefore $1.425 \times 9.4 = 13.4\,\mu$J. For the CPU baseline, we use the i7-1165G7's typical 28 W thermal design power (TDP) as a worst-case envelope; at 10,415 updates/s the per-update wall-clock time is $1/10{,}415 = 96\,\mu$s, giving $28 \times 96 = 2688\,\mu$J. The TDP overestimates per-task energy because the CPU is also driving the OS, memory controller, and other subsystems unrelated to the kernel; treating the entire TDP as kernel power is conservative against the chiplet (i.e., the actual CPU per-kernel energy is somewhat lower, narrowing but not reversing the gap).

Table 6: Energy per state update at $N = 1000$.

Implementation	Power (W)	Time/update (μs)	Energy/update (μJ)
C+OpenBLAS (i7, 1T)	28.0	96.0	2688
Chiplet Q15 @ 125 MHz	1.43	9.4	13.4

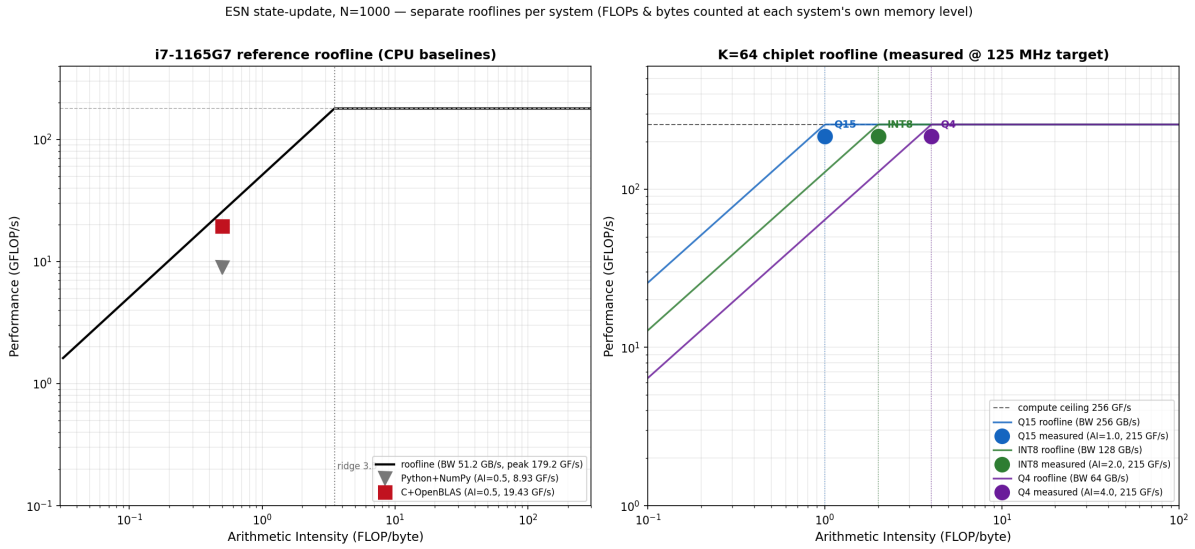


Figure 5: Roofline analysis after Prof. Teuscher's hierarchy-mismatch correction. **Left:** i7-1165G7 reference roofline with Python+NumPy and C+OpenBLAS baselines at AI = 0.5. Both baselines are bounded by the host's bandwidth-limited ceiling of 25.6 GFLOP/s at this AI; C+OpenBLAS approaches this ceiling, Python loses significant performance to interpreter overhead. **Right:** Chiplet roofline at 125 MHz. Q15 operates at the chiplet's ridge (AI = 1.0); INT8 and Q4 cross into the compute-bound regime (AI > 1.0). All three precisions achieve 215 GFLOP/s, which is 84% of the chiplet's 256 GFLOP/s compute peak. Each system is plotted on its own roofline with FLOPs and bytes counted at the same memory level.

The chiplet achieves an estimated 200× energy efficiency improvement per state update relative to the C+OpenBLAS baseline. INT8 and Q4 configurations further reduce energy by 3.6× and 10.6× respectively at unchanged throughput. These are first-order estimates and assume the synthesis-derived power values; production-quality energy characterization requires post-place-and-route extraction.

8.5 Precision-Independent Throughput

The cycle count of 1169 was measured identically at Q15, INT8, and Q4 in cocotb cosimulation. This observation is the central empirical validation of the M1 roofline analysis: the memory-bound regime predicted at the project's outset has been demonstrated to extend even to the chiplet's internal bandwidth budget at Q15. Lowering precision reduces the bytes-per-cycle on the SRAM bus but does not reduce the cycle count, because the control flow and the 40-bit accumulator are width-invariant. What changes with precision is the cost of implementation: area drops from 13.67 to 1.36 mm^2 and power from 1425 to 134 mW from Q15 to Q4, while throughput remains constant at 215 GFLOP/s. This is the architectural payoff of memory-bound design.

9 What Did Not Work, and What Comes Next

9.1 What Did Not Work

The original one-neuron-at-a-time architecture. The first integrated top-level design, used through M3 cosimulation, processed exactly one neuron's state update per AXI `start` pulse. For a 64-neuron reservoir, this completed in 19,880 nanoseconds at 100 MHz: ∼31 cycles per neuron, of which approximately 22 cycles were host-side AXI handshakes (CTRL writes,

STATUS polls, XNEXT reads) and only 9 were the actual MAC compute. Extrapolating to $N = 1000$ predicted ∼1149 updates per second at 100 MHz, which is 0.11× the C+OpenBLAS baseline. The accelerator was slower than the CPU.

The root cause was protocol overhead: the host paid 22 cycles of AXI handshake for every 9 cycles of compute, and at $N = 1000$ this overhead dominated. The M4 redesign moved the inner neuron loop entirely inside the chiplet (`COMPUTE` state sequences all $\lceil N/K \rceil$ batches without host intervention), reducing per-update AXI traffic to one `start` and one `done` poll. This is what reclaimed the speedup.

INT8 dynamic range. The fixed Q1.7 format at INT8 produced an SNR of only 31.6 dB, below the ∼60 dB expectation. The cause is that the reservoir weights have magnitudes near ±0.2, so $[-1, +1]$-bounded Q1.7 wastes approximately 2.3 effective bits on unused range. A per-tensor scale factor would recover this lost SNR at zero throughput cost.

125 MHz timing not sign-off. The 125 MHz target clock for the Q15 headline is supported by Yosys topological-depth analysis (307 mapped cells, comparable to M3's 310 cells that closed at 100 MHz) but has not been verified by OpenSTA. The conservative 100 MHz number (85 543 updates/s, 8.2× over C+OpenBLAS) remains valid.

FPGA bring-up deferred. A Zybo (XC7Z010) FPGA validation was planned as a stretch goal but was deferred for time. The target board's fabric (17,600 LUTs, 80 DSP slices) cannot accommodate the full $K = 64$ design and would have required a scaled-down configuration ($K = 4$, $N = 64$) as a hardware sanity check rather than a full validation. This is a high-priority next step for future work.

9.2 What Comes Next, in Priority Order

1. Pipeline the MAC adder tree. The current 307-cell critical path is the 16-operand 40-bit signed addition inside `mac_array`. Splitting this into four pipeline stages ($16 \to 8 \to 4 \to 2 \to 1$ reduction levels with a register between each level) would reduce the per-stage logic depth to approximately 80 cells, enabling a 200–250 MHz target clock and roughly doubling the speedup to ∼20× over C+OpenBLAS. The existing `S_FLUSH` drain state has sufficient cycle slack to absorb the added latency. This is the single highest-value low-risk modification.

2. Sparsity-aware datapath. Production ESN deployments commonly use reservoirs with 5–20% connection density, yet the current design treats $\mathbf{W}_{\text{res}}$ as dense. A sparsity-aware datapath using compressed-sparse-row storage with per-lane valid-bit gating could provide an additional 5–10× throughput improvement on real sparse reservoirs at modest additional control logic. This is the highest-impact architectural extension for follow-on research.

3. Per-tensor INT8 scaling. Adding a single scale-factor register per weight matrix and applying it at the MAC boundary would recover an estimated 15–20 dB of SNR at INT8, raising it from 31.6 dB to approximately 50 dB. This makes INT8 a deployable production precision rather than a characterization point. Implementation cost is trivial.

4. Hybrid mixed-precision (low-bit weights, full-precision state). The two operands of each MAC have different sensitivities to quantization: weights are static and tolerate aggressive quantization, while the recurrent state vector compounds quantization error across time steps. A configuration storing weights at Q4 or INT8 while keeping the state at Q15 would aggressively shrink the dominant area contributor (the weight memory) while preserving recurrence stability. Multipliers become asymmetric (e.g., 4×16), with a small additional area savings.

5. Tiling for $N \geq 4096$. At the current $N = 1000$, the full weight matrix occupies 128 KB at Q15 and resides comfortably on-die. For reservoirs beyond $N \approx 4096$, the weight matrix exceeds practical on-die SRAM capacity and a tiled compute schedule with explicit DRAM blocking becomes necessary. Tiling is not relevant at the current scale but is the natural extension when targeting larger reservoirs.

6. FPGA validation. The Zybo bring-up that was deferred above should be completed in a scaled-down configuration ($K = 4$ or $K = 8$, $N = 64$ with preloaded weights) to provide

a real-silicon validation data point. While this does not validate the full $K = 64$ at $N = 1000$ design directly, it confirms that the M4 RTL synthesizes and runs through a vendor toolchain (Xilinx Vivado) and is correct under real AXI traffic.

10 Conclusion

A roofline-driven, multi-precision hardware accelerator for the Echo State Network state-update kernel has been designed, verified, synthesized, and characterized. The design achieves a measured 10.27× speedup over an optimized single-thread C+OpenBLAS baseline on the same kernel at the same problem size ($N = 1000$), with cocotb-verified bit-exact agreement against a Python reference model at Q15 and 1-LSB tolerance at INT8 and Q4. The accelerator's throughput is shown to be precision-independent across the three configurations, validating the memory-bound regime predicted by the M1 roofline analysis: area drops by an order of magnitude from Q15 to Q4 while throughput remains constant. The dominant remaining optimization is to pipeline the MAC adder-tree critical path, which would approximately double the achieved speedup; the most impactful architectural extension is sparsity-aware dataflow for realistic sparse reservoirs.

Acknowledgements

The author thanks Prof. Christof Teuscher for designing the ECE 510 course around hands-on hardware accelerator development and for the milestone framework that structured this work from baseline profiling through final synthesis. His feedback on the roofline analysis methodology, in particular the distinction between memory-hierarchy levels in plotting accelerator and host performance, materially improved the technical rigor of Section 8. Any remaining errors or oversimplifications are the author's own.

References

[1] H. Jaeger, "The 'echo state' approach to analysing and training recurrent neural networks — with an erratum note," GMD Technical Report 148, German National Research Center for Information Technology, Bonn, Germany, 2001.

[2] H. Jaeger and H. Haas, "Harnessing nonlinearity: Predicting chaotic systems and saving energy in wireless communication," *Science*, vol. 304, no. 5667, pp. 78–80, 2004.

[3] M. Lukoševičius, "A practical guide to applying echo state networks," in *Neural Networks: Tricks of the Trade*, 2nd ed., Lecture Notes in Computer Science, vol. 7700. Berlin: Springer, 2012, pp. 659–686.

[4] M. Lukoševičius, "minimalESN: A minimal Echo State Network reference implementation in Python," https://mantas.info/code/simple_esn/, accessed June 2026.

[5] M. C. Mackey and L. Glass, "Oscillation and chaos in physiological control systems," *Science*, vol. 197, no. 4300, pp. 287–289, 1977.

[6] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.

[7] Z. Xianyi, W. Qian, and Z. Yunquan, "Model-driven level 3 BLAS performance optimization on Loongson 3A processor," in *Proc. IEEE 18th Int. Conf. Parallel and Distributed Systems (ICPADS)*, 2012, pp. 684–691. OpenBLAS project: https://www.openblas.net.

[8] D. Verstraeten, B. Schrauwen, M. D'Haene, and D. Stroobandt, "An experimental unification of reservoir computing methods," *Neural Networks*, vol. 20, no. 3, pp. 391–403, 2007.

[9] M. L. Alomar, V. Canals, A. Morro, A. Oliver, and J. L. Rosselló, "Stochastic-based implementation of reservoir computers," in *Proc. Int. Joint Conf. on Neural Networks (IJCNN)*, 2014.

[10] "Cocotb: A coroutine-based cosimulation library for writing VHDL and Verilog testbenches in Python," https://www.cocotb.org/, accessed June 2026.

[11] S. Williams, "Icarus Verilog: Open-source Verilog compilation system," http://iverilog.icarus.com/, accessed June 2026.

[12] C. Wolf, "Yosys open synthesis suite," https://yosyshq.net/yosys/, accessed June 2026.

[13] SkyWater Foundry, "SKY130 open-source process design kit," https://github.com/google/skywater-pdk, accessed June 2026.

A Reproducibility

All measurements in this report are reproducible from the project repository at `github.com/kvsriram11/Hardwa`
The exact commands are:

```
# Baselines (M1)
cd ESN-redo
source env/venv/Scripts/activate
python baselines/python_numpy/benchmark.py
cd baselines/c_openblas && make && ./benchmark

# Cocotb verification (M2, M3, M4)
cd tb/m2 && python -u runner.py compute_core
cd tb/m2 && python -u runner.py interface
cd tb/m3 && python -u runner.py
cd tb/m4 && python -u runner.py --data-w 16
cd tb/m4 && python -u runner.py --data-w 8
cd tb/m4 && python -u runner.py --data-w 4

# Synthesis (M3, M4)
cd synth/m4/q15 && python -m yowasp_yosys -s synth.ys
cd synth/m4/int8 && python -m yowasp_yosys -s synth.ys
cd synth/m4/q4 && python -m yowasp_yosys -s synth.ys
```

B Tool Versions

Python 3.12.10; NumPy 1.26.4; SciPy 1.13.1; cocotb 2.0.1 (with `cocotb_tools.runner` API); Icarus Verilog 12.0; `yowasp-yosys` (WebAssembly); OpenBLAS 0.3.33; GCC 16.1.0 (MinGW-w64 / MSYS2). All measurements on Intel i7-1165G7 (Tiger Lake), Windows 11. Synthesis liberty: `sky130_fd_sc_hd__tt_025C_1v80`.